



GENE STORM

Game Design Document

Djih

October 4, 2025

Contents

1	Overview	3
1.1	Summary	3
1.2	Vision	3
1.3	Scope	3
1.4	Glossary	3
2	Core Loop	3
2.1	Game Progression	3
2.2	Player Interaction	3
2.3	Reward an Motivations	3
2.4	Content Roadmap	3
3	System Specifications	3
3.1	Stats and Formulas	3
3.2	Fighting System	3
3.3	Skill System	3
3.4	Items and Inventory	3
3.5	Perks and Traits	3
3.6	Enemies	3
3.7	Loot and Rarities	3
3.8	Loot-Roll Timeline (End-to-End)	3
3.8.1	Begriffe Ziele	3
3.8.2	0) Vor der Welle	4
3.8.3	1) Pro Gegner-Kill → Rolls	4
3.8.4	2) Drop-Pool iterieren	4
3.8.5	3) Einzelwurf – Faktor-Reihenfolge	4
3.8.6	4) Erfolg Buchung	4
3.8.7	5) Secret-Drops (optional, zusätzlich)	5
3.8.8	6) Wellenabschluss – Failsafes Garantien	5
3.8.9	7) Hinweise zur Balance	5

3.8.10 8) Telemetrie (für Tuning)	5
3.9 Meta Progression	5
4 Content Bible	5
4.1 Archetypes and Characters	5
4.2 Skill Catalog	5
4.3 Modifier Catalog	5
4.4 Enemy Taxonomy	5
4.5 Maps Waves	5
5 Technical Design	5
5.1 Architecture Overview	5
5.2 Datamodell	6
5.3 Runtime-Pipelines	6
5.4 Saving	6
5.5 Tooling and Debugging	6
5.6 Performance Budget	6
5.7 Coding Guidelines	6
6 UI/UX Specification	6
6.1 Information Flow	6
6.2 Key Screens	6
6.3 Input Periphery	6
6.4 Feedback	6
7 Balance and Tuning	6
7.1 Target Values	6
7.2 Curves	6
7.3 Test Scenarios	6



1 Overview

1.1 Summary

1.2 Vision

1.3 Scope

1.4 Glossary

2 Core Loop

2.1 Game Progression

2.2 Player Interaction

2.3 Reward and Motivations

2.4 Content Roadmap

3 System Specifications

3.1 Stats and Formulas

3.2 Fighting System

3.3 Skill System

3.4 Items and Inventory

3.5 Perks and Traits

3.6 Enemies

3.7 Loot and Rarities

3.8 Loot Roll Logic (End-to-End)

Purpose

This section specifies the complete loot flow from wave setup to final rewards. It is implementation-agnostic and defines inputs, order of operations, and balancing knobs.

Key Terms

- **Wave Budget** B : Total loot value budget allocated to a wave.
- **Spent Budget** U : Accumulated loot value already awarded in the current wave.
- **Rarity** R : Item quality tier (e.g., Common, Uncommon, Rare, Epic, Legendary, ...).
- **Base Chance** $p_{\text{base}}[R]$: The configured per-rarity drop chance before any modifiers.



- **Unlucky Multiplier** $m_{\text{unlucky}}[R]$: Pity-protection factor derived from recent fails / hits per rarity.
- **Luck Factor** L : Linear luck strength; $L = 0$ (no bonus), $L = 1$ (+100)
- **Luck Scale** $k[R]$: Optional per-rarity scaling of luck (default $k[R] = 1.0$).
- **Budget Multiplier** $m_{\text{budget}}(U, \text{lootValue}, B, R)$: Soft damping when U approaches/exceeds B , including rarity floors.
- **Effective Tags**: The union of an enemy's *baseTags* and *bonusTags*.

0) Wave Setup

1. **Compute Wave Budget** B : Derived from wave composition (ranks, counts), map difficulty and level; add variance; initialize $U = 0$.
2. **Load and Merge Loot Rules**: For all tags present in the wave, prepare the merged rule set (drop pools, min/max drops, rarity guarantees, secret rules).

1) On Enemy Kill

1. **Determine Roll Count**: Based on enemy rank/type (e.g., Elite may trigger more rolls than Normal).
2. **Build Effective Tag Set**: $\text{effectiveTags} = \text{baseTags} \cup \text{bonusTags}$ (duplicates ignored for logic; weighting optional).

2) Per Roll (for this enemy)

1. **Pick Tag**: Randomly select one tag from *effectiveTags* (optionally bias *bonusTags* slightly).
2. **Resolve Tag Rule**: Activate the corresponding merged loot rule for this tag (yields a list of loot entries to attempt).

3) Iterate the Tag's Drop Entries

For each *loot entry* (which may be a single chance or a list of chances), evaluate one *rarity block* at a time. The system supports *parallel rarity rolls* so that multiple rarities can succeed in the same slot.

Factor Order (per rarity R):

1. **Base Chance**: $p_{\text{base}}[R]$
2. **Unlucky (Pity)**: multiply by $m_{\text{unlucky}}[R]$
3. **Luck (Linear, No Cap)**: multiply by $(1 + L \cdot k[R])$ *Example*: $p_{\text{eff, preBudget}}[R] = p_{\text{base}}[R] \cdot m_{\text{unlucky}}[R] \cdot (1 + L \cdot k[R])$
4. **Budget (Soft Brake + Floors)**: multiply by $m_{\text{budget}}(U, \text{lootValue}, B, R) \rightarrow p_{\text{roll}}[R] = p_{\text{eff, preBudget}}[R] \cdot m_{\text{budget}}(\cdot)$
5. **RNG Roll**: If success, **award one item** of rarity R (this slot can yield multiple items if several rarities succeed).



Post-Roll Accounting (per rarity):

- On success: add the item to results; increase U by its `lootValue`; call `Unlucky.OnDrop( $R$ )`.
- On fail: call `Unlucky.OnFail( $R$ )`.

4) Secret Drops (Optional, Per Enemy)

Evaluate secret/conditional drops (e.g., specific tag combinations). These are additional rolls that can yield extra items. They do not interfere with the regular pity counters unless explicitly specified.

5) Wave Finalization

1. **MinDrops Failsafe:** If total items $<$ configured minimum for the wave, add items from the wave's mixed pool until the minimum is met. This path increases U ; whether it resets pity per rarity is a policy toggle (recommended: *does not* reset pity to preserve fairness).
2. **Rarity Guarantees:** If a minimum per rarity is required, top up those rarities. This path typically *does not* alter pity and may optionally bypass budget damping depending on economy policy.

Design Notes

- **Parallel Rarity Rolls:** Each rarity is evaluated independently; *all successes drop*. Lower-tier success does not consume or block higher-tier success.
- **Luck Scope:** Luck affects *quality* (rarity chances) only. Quantity remains governed by entries, rules, and budget. If desired, allow certain luck-driven drops to *ignore* budget damping (economy-sensitive).
- **Budget as Soft Brake:** m_{budget} smoothly reduces chances when U exceeds B ; rarity floors prevent near-zero results for higher tiers.
- **Tags as Theme Driver:** Use *effectiveTags* for both regular and secret rolls; optional small biases can keep `bonusTags` slightly more impactful without excluding `baseTags`.

Balancing Knobs

- **Per-rarity base chances** $p_{\text{base}}[R]$
- **Unlucky growth/reset rates** per rarity
- **Luck factor** L (from passive tree, map mods, etc.) and **luck scales** $k[R]$
- **Budget damping** curve parameters and **rarity floors**
- **Tag pick weighting** (base vs. bonus) and secret-drop criteria
- **Failsafe policies:** whether MinDrops/Guarantees affect pity or budget



3.9 Meta Progression

4 Content Bible

4.1 Archetypes and Characters

4.2 Skill Catalog

4.3 Modifier Catalog

4.4 Enemy Taxonomy

4.5 Maps Waves

5 Technical Design

5.1 Architecture Overview

5.2 Datamodell

5.3 Runtime-Pipelines

5.4 Saving

5.5 Tooling and Debugging

5.6 Performance Budget

5.7 Coding Guidelines

6 UI/UX Specification

6.1 Information Flow

6.2 Key Screens

6.3 Input Periphery

6.4 Feedback

7 Balance and Tuning

7.1 Target Values

7.2 Curves

7.3 Test Scenarios